

LAPORAN PRAKTIKUM PEKAN 8

STRUKTUR DATA

Disusun Oleh:

Muhammad Fharel

2511531010

Dosen Pengampu:

Dr. Wahyudi S.T.M.T

Asisten Pratikum:

Muhammad Galid Avero



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Segala puji penulis panjatkan ke hadirat Allah SWT atas rahmat dan karunia-Nya sehingga laporan praktikum Struktur Data pada tanggal 2 Juni 2026 yang membahas beberapa algoritma sorting yang umum digunakan, yaitu Shell Sort, Quick Sort, Merge Sort, serta implementasi visualisasi Bubble Sort dan Merge Sort menggunakan Graphical User Interface (GUI). Melalui praktikum ini, penulis dapat memahami bagaimana setiap algoritma bekerja dalam mengurutkan data serta melihat proses pengurutan secara lebih interaktif melalui tampilan GUI.

Ucapan terima kasih ditujukan kepada dosen pengampu, asisten praktikum, serta rekan-rekan yang telah membantu dalam proses pelaksanaan praktikum. Penulis menyadari bahwa penulisan laporan masih jauh dari kata sempurna, oleh karena itu kritik dan saran sangat diharapkan untuk penyempurnaan di kemudian hari. Semoga laporan ini memberikan manfaat dan menambah wawasan pembaca.

Padang, 4 Juni 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang.....	1
1.2 Tujuan Praktikum	1
1.3 Manfaat Praktikum	1
BAB II PEMBAHASAN	3
2.1 Sorting Algorithm.....	3
2.2 Kode Pemograman	4
2.2.1 Class Shell Sort	4
2.2.2 Output Shell Sort.....	5
2.2.3 Class Quick Sort.....	6
2.2.4 Output Quick Sort	7
2.2.5 Class Merge Sort.....	8
2.2.6 Output Merge Sort.....	9
2.2.7 Class Bubble Sort GUI.....	10
2.2.8 Output Bubble Sort GUI	15
2.2.9 Class Merge Sort GUI.....	16
2.2.10 Output Merge Sort GUI	21
BAB III PENUTUP	23
3.1 Kesimpulan.....	23
DAFTAR PUSTAKA	24
TUGAS PRAKTIKUM.....	25

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam pemrograman, proses pengurutan data merupakan salah satu operasi yang sangat penting karena dapat mempermudah pencarian, pengolahan, dan pengelolaan informasi. Semakin banyak data yang diproses, semakin dibutuhkan algoritma sorting yang efisien agar waktu eksekusi program menjadi lebih cepat.

Selain algoritma sorting sederhana seperti Bubble Sort dan Insertion Sort, terdapat algoritma yang lebih efisien seperti Shell Sort, Quick Sort, dan Merge Sort. Algoritma-algoritma tersebut memiliki mekanisme yang berbeda dalam mengurutkan data dan mampu memberikan performa yang lebih baik pada jumlah data yang besar.

Pada praktikum ini dilakukan implementasi berbagai algoritma sorting tersebut serta visualisasi proses Bubble Sort dan Merge Sort menggunakan GUI. Dengan demikian, mahasiswa dapat memahami tidak hanya hasil akhir pengurutan, tetapi juga proses yang terjadi pada setiap langkah algoritma.

1.2 Tujuan Praktikum

Adapun tujuan dari praktikum ini adalah:

1. Mengetahui cara kerja Shell Sort, Quick Sort, dan Merge Sort.
2. Mengimplementasikan algoritma sorting menggunakan Java.
3. Memahami proses sorting secara bertahap melalui GUI.
4. Membandingkan cara kerja beberapa metode sorting yang berbeda.

1.3 Manfaat Praktikum

Adapun tujuan dari praktikum ini adalah:

1. Menambah pemahaman tentang algoritma pengurutan data.
2. Melatih kemampuan implementasi algoritma dalam Java.
3. Memahami kelebihan dan karakteristik masing-masing metode sorting.
4. Mempermudah pembelajaran melalui visualisasi GUI.
5. Menjadi dasar untuk mempelajari algoritma yang lebih kompleks.

BAB II

PEMBAHASAN

2.1 Sorting Algorithm

Sorting merupakan proses mengurutkan data berdasarkan urutan tertentu, seperti dari kecil ke besar atau sebaliknya. Dalam pemrograman Java, algoritma sorting digunakan untuk membantu proses pengolahan data agar lebih terstruktur dan mudah diakses.

Pada praktikum ini digunakan tiga metode sorting, yaitu Insertion Sort, Selection Sort, dan Bubble Sort. Ketiga algoritma tersebut memiliki cara kerja yang berbeda, namun tujuan utamanya sama yaitu mengurutkan data.

a. Shell Sort

Shell Sort merupakan pengembangan dari Insertion Sort yang menggunakan konsep gap atau jarak antar elemen. Pada awal proses, data dibandingkan menggunakan jarak tertentu, kemudian jarak tersebut diperkecil secara bertahap hingga menjadi satu. Dengan cara ini, perpindahan data dapat dilakukan lebih cepat sehingga proses sorting menjadi lebih efisien dibandingkan Insertion Sort biasa.

b. Quick Sort

Quick Sort merupakan algoritma sorting yang menggunakan teknik divide and conquer. Data dibagi berdasarkan sebuah pivot, kemudian elemen yang lebih kecil ditempatkan di sebelah kiri pivot dan elemen yang lebih besar ditempatkan di sebelah kanan. Pada program ini digunakan metode Median of Three untuk menentukan pivot sehingga hasil pembagian data menjadi lebih optimal. Setelah itu proses dilakukan secara rekursif hingga seluruh data terurut.

c. Merge Sort

Merge Sort juga menggunakan teknik divide and conquer. Array dibagi menjadi dua bagian yang lebih kecil secara berulang hingga tersisa satu elemen pada setiap bagian. Setelah itu dilakukan proses penggabungan kembali (merge) dengan membandingkan elemen-elemen yang ada sehingga menghasilkan array yang sudah terurut. Algoritma ini terkenal stabil dan memiliki performa yang baik pada data berukuran besar.

Selain implementasi dalam bentuk program biasa, praktikum ini juga menggunakan GUI untuk memvisualisasikan proses Bubble Sort dan Merge Sort secara langkah demi langkah. Dengan adanya tampilan visual, proses perpindahan data menjadi lebih mudah dipahami dibandingkan hanya melihat output teks biasa.

2.2 Kode Pemograman

2.2.1 Class Shell Sort

```
2 package pekan8_2511531010;
3
4 public class ShellSort_2511531010 {
5     public static void shellSort(int[] A) {
6         int n_1010 = A.length;
7         int gap_1010 = n_1010 / 2;
8         while (gap_1010 > 0) {
9             for (int i_1010 = gap_1010; i_1010 < n_1010;
10 i_1010++) {
11                 int temp_1010 = A[i_1010];
12                 int j_1010 = i_1010;
13                 while (j_1010 >= gap_1010 && A[j_1010 -
14 gap_1010] > temp_1010) {
15                     A[j_1010] = A[j_1010 - gap_1010];
16                     j_1010 = j_1010 - gap_1010;
17                 }
18                 A[j_1010] = temp_1010;
19             }
20             gap_1010 = gap_1010 / 2;
21         }
22
23     public static void main(String[] args) {
24         int[] data_1010 = {3, 10, 4, 6, 8, 9, 7, 2, 1, 5};
25
26         System.out.print("Sebelum: ");
```

```

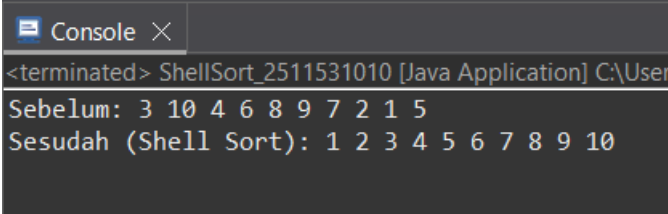
27     printArray(data_1010);
28
29     shellSort(data_1010);
30
31     System.out.print("Sesudah (Shell Sort): ");
32     printArray(data_1010);
33 }
34
35 public static void printArray(int[] arr) {
36     for (int i_1010 : arr) System.out.print(i_1010 + " ");
37     System.out.println();
38 }
39 }

```

Gambar 2.1

Proses sorting dilakukan dengan membandingkan elemen-elemen yang memiliki jarak tertentu (gap). Nilai gap awal ditentukan setengah dari panjang array, kemudian terus diperkecil hingga bernilai 1. Dengan cara ini, data yang berada jauh dari posisi seharusnya dapat dipindahkan lebih cepat dibandingkan Insertion Sort biasa. Setelah gap menjadi 1, proses yang terjadi mirip seperti Insertion Sort sehingga menghasilkan data yang telah terurut secara sempurna.

2.2.2 Output Shell Sort



```

<terminated> ShellSort_2511531010 [Java Application] C:\User
Sebelum: 3 10 4 6 8 9 7 2 1 5
Sesudah (Shell Sort): 1 2 3 4 5 6 7 8 9 10

```

Gambar 2.2

Output program menampilkan dua kondisi data, yaitu sebelum dan sesudah proses sorting. Pada awalnya data masih berada dalam urutan acak. Setelah algoritma Shell Sort dijalankan, seluruh angka tersusun secara berurutan dari nilai terkecil hingga terbesar. Hasil tersebut menunjukkan bahwa proses pengurangan gap dan perpindahan elemen berhasil mengelompokkan data ke posisi yang tepat sehingga array menjadi terurut.

2.2.3 Class Quick Sort

```
2 package pekan8_2511531010;
3
4 public class QuickSort_2511531010 {
5     static void swap(int[] arr_1010, int i_1010, int j_1010) {
6         int temp_1010 = arr_1010[i_1010];
7         arr_1010[i_1010] = arr_1010[j_1010];
8         arr_1010[j_1010] = temp_1010;
9     }
10    // Metode tambahan untuk mengatur pivot menggunakan Median-of-
    Three
11    static void medianOfThree (int[] arr_1010, int low_1010, int
    high_1010) {
12        int mid_1010 = low_1010 + (high_1010 - low_1010) / 2;
13
14        //Urutkan elemen low, mid, dan high
15        if (arr_1010[low_1010] > arr_1010[mid_1010]) {
16            swap(arr_1010, low_1010, mid_1010);
17        }
18        if (arr_1010[low_1010] > arr_1010[high_1010]) {
19            swap(arr_1010, low_1010, high_1010);
20        }
21        if (arr_1010[mid_1010] > arr_1010[high_1010]) {
22            swap(arr_1010, mid_1010, high_1010);
23        }
24        swap(arr_1010, mid_1010, high_1010);
25    }
26    static int partition(int[] arr_1010, int low_1010, int high_1010)
    {
27        // Panggil fungsi medianOfThree sebelum menentukan pivot
28        medianOfThree(arr_1010, low_1010, high_1010);
29
30        int pivot_1010 = arr_1010[high_1010]; // Sekarang arr[high]
    sudah berisi nilai median
31        int i_1010 = (low_1010 - 1);
32
33        for (int j_1010 = low_1010; j_1010 <= high_1010 - 1;
    j_1010++) {
34            // Jika elemen saat ini lebih kecil dari atau sama
    dengan pivot
35            if (arr_1010[j_1010] <= pivot_1010) {
36                // Increment indeks elemen yang lebih kecil
37                i_1010++;
38                swap(arr_1010, i_1010, j_1010);
39            }
40        }
41        swap(arr_1010, i_1010 + 1, high_1010);
42        return (i_1010 + 1);
43    }
44    static void quickSort(int[] arr_1010, int low_1010, int
    high_1010) {
45        if (low_1010 < high_1010) {
46            int pi_1010 = partition(arr_1010, low_1010,
    high_1010);
```

```

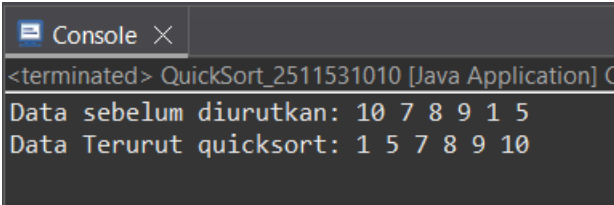
47         quickSort(arr_1010, low_1010, pi_1010 - 1);
48         quickSort(arr_1010, pi_1010 + 1, high_1010);
49     }
50 }
51
52 public static void printArray(int[] arr_1010) {
53     for (int i_1010 = 0; i_1010 < arr_1010.length; i_1010++) {
54         System.out.print(arr_1010[i_1010] + " ");
55     }
56     System.out.println();
57 }
58
59 public static void main(String[] args) {
60     int[] arr_1010 = { 10, 7, 8, 9, 1, 5 };
61     int N_1010 = arr_1010.length;
62     System.out.print("Data sebelum diurutkan: ");
63     printArray(arr_1010);
64
65     quickSort(arr_1010, 0, N_1010 - 1);
66
67     System.out.print("Data Terurut quicksort: ");
68     printArray(arr_1010);
69 }
70 }

```

Gambar 2.3

Program memilih sebuah nilai pivot menggunakan metode Median of Three, kemudian membagi data menjadi dua kelompok, yaitu data yang lebih kecil dari pivot dan data yang lebih besar dari pivot. Setelah proses pembagian selesai, masing-masing kelompok diurutkan kembali secara rekursif hingga seluruh data tersusun dengan benar

2.2.4 Output Quick Sort



```

Console ×
<terminated> QuickSort_2511531010 [Java Application] C
Data sebelum diurutkan: 10 7 8 9 1 5
Data Terurut quicksort: 1 5 7 8 9 10

```

Gambar 2.4

Output program memperlihatkan kondisi data sebelum dan sesudah proses pengurutan. Setelah Quick Sort dijalankan, seluruh angka yang sebelumnya tidak beraturan berubah menjadi urutan yang rapi dari kecil ke besar. Hasil ini menunjukkan bahwa proses pemilihan pivot dan

pembagian data berjalan dengan baik sehingga setiap elemen dapat ditempatkan pada posisi yang sesuai.

2.2.5 Class Merge Sort

```
2 package pekan8_2511531010;
3
4 public class MergeSort_2511531010 {
5     void merge(int arr_1010[], int l_1010, int m_1010, int r) {
6         //Find sizes of two subarrays to be merged
7         int n1_1010 = m_1010 - l_1010 + 1;
8         int n2_1010 = r - m_1010;
9         /* Create temp arrays */
10        int L_1010[] = new int[n1_1010];
11        int R_1010[] = new int[n2_1010];
12        /* Copy data to temp arrays */
13        for (int i_1010 = 0; i_1010 < n1_1010; ++i_1010)
14            L_1010[i_1010] = arr_1010[l_1010 + i_1010];
15        for (int j_1010 = 0; j_1010 < n2_1010; ++j_1010)
16            R_1010[j_1010] = arr_1010[m_1010 + 1 + j_1010];
17        int i_1010 = 0, j_1010 = 0;
18        // Initial index of merged subarray array
19        int k_1010 = l_1010;
20        while (i_1010 < n1_1010 && j_1010 < n2_1010) {
21            if (L_1010[i_1010] <= R_1010[j_1010]) {
22                arr_1010[k_1010] = L_1010[i_1010];
23                i_1010++;
24            } else {
25                arr_1010[k_1010] = R_1010[j_1010];
26                j_1010++;
27            }
28            k_1010++;
29        }
30        /* Copy remaining elements of L[] if any */
31        while (i_1010 < n1_1010) {
32            arr_1010[k_1010] = L_1010[i_1010];
33            i_1010++;
34            k_1010++;
35        }
36        /* Copy remaining elements of R[] if any */
37        while (j_1010 < n2_1010) {
38            arr_1010[k_1010] = R_1010[j_1010];
39            j_1010++;
40            k_1010++;
41        }
42    }
43    void sort(int arr_1010[], int l_1010, int r_1010) {
44        if (l_1010 < r_1010) {
45            // Find the middle point
46            int m_1010 = (l_1010 + r_1010) / 2;
47            // Sort first and second halves
48            sort(arr_1010, l_1010, m_1010);
```

```

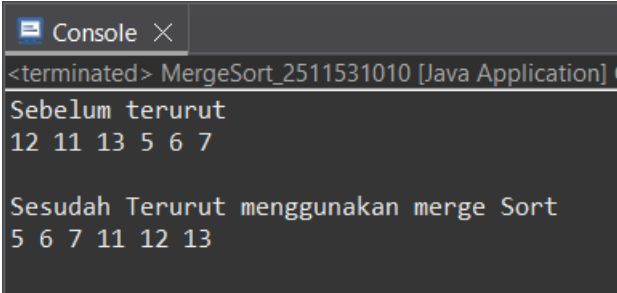
49         sort(arr_1010, m_1010 + 1, r_1010);
50         // Merge the sorted halves
51         merge(arr_1010, l_1010, m_1010, r_1010);
52     }
53 }
54 /* A utility function to print array of size n */
55 static void printArray(int arr_1010[]) {
56     int n_1010 = arr_1010.length;
57     for (int i_1010 = 0; i_1010 < n_1010; ++i_1010)
58         System.out.print(arr_1010[i_1010] + " ");
59     System.out.println();
60 }
61
62 public static void main(String args[]) {
63     int arr_1010[] = { 12, 11, 13, 5, 6, 7 };
64     System.out.println("Sebelum terurut");
65     printArray(arr_1010);
66
67     MergeSort_2511531010 ob_1010 = new MergeSort_2511531010();
68     ob_1010.sort(arr_1010, 0, arr_1010.length - 1);
69
70     System.out.println("\nSesudah Terurut menggunakan merge
Sor");
71     printArray(arr_1010);
72 }
73 }

```

Gambar 2.5

Class ini mengimplementasikan algoritma Merge Sort yang bekerja dengan membagi array menjadi beberapa bagian kecil. Setiap bagian kemudian diurutkan secara terpisah dan digabungkan kembali menggunakan method `merge()`. Saat proses penggabungan berlangsung, elemen-elemen dari dua kelompok yang sudah terurut dibandingkan satu per satu untuk menghasilkan susunan data yang benar.

2.2.6 Output Merge Sort



```

Console X
<terminated> MergeSort_2511531010 [Java Application]
Sebelum terurut
12 11 13 5 6 7

Sesudah Terurut menggunakan merge Sort
5 6 7 11 12 13

```

Gambar 2.6

Output menunjukkan perubahan data dari kondisi acak menjadi terurut. Hasil akhir membuktikan bahwa proses pembagian array dan penggabungan kembali berhasil dilakukan dengan benar. Data yang semula tersebar pada posisi yang tidak beraturan akhirnya tersusun dari nilai terkecil hingga terbesar setelah seluruh tahapan merge selesai.

2.2.7 Class Bubble Sort GUI

```
2 package pekan8_2511531010;
3
4 import java.awt.BorderLayout;
5 import java.awt.Color;
6 import java.awt.Dimension;
7 import java.awt.FlowLayout;
8 import java.awt.Font;
9 import javax.swing.BorderFactory;
10 import javax.swing.JButton;
11 import javax.swing.JFrame;
12 import javax.swing.JLabel;
13 import javax.swing.JOptionPane;
14 import javax.swing.JPanel;
15 import javax.swing.JScrollPane;
16 import javax.swing.JTextArea;
17 import javax.swing.JTextField;
18 import javax.swing.SwingConstants;
19 import javax.swing.SwingUtilities;
20
21 public class BubbleSortGUI_2511531010 extends JFrame {
22     private static final long serialVersionUID = 1L;
23     private int[] array_1010;
24     private JLabel[] labelArray_1010;
25     private JButton stepButton_1010, resetButton_1010,
26     setButton_1010;
27     private JTextField inputField_1010;
28     private JPanel panelArray_1010;
29     private JTextArea stepArea_1010;
30
31     private int i_1010 = 0;
32     private int j_1010 = 0;
33     private int stepCount_1010 = 1;
34     private boolean sorting_1010 = false;
35
36     /**
37      * Create the frame.
38      */
39     public BubbleSortGUI_2511531010() {
40         setTitle("Bubble Sort Langkah per Langkah");
41         setSize(750, 400);
42         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
```

```

42         setLocationRelativeTo(null);
43         setLayout(new BorderLayout());
44
45         // Panel input
46         JPanel inputPanel_1010 = new JPanel(new FlowLayout());
47         inputField_1010 = new JTextField(30);
48         setButton_1010 = new JButton("Set Array");
49         inputPanel_1010.add(new JLabel("Masukkan angka (pisahkan
dengan koma):"));
50         inputPanel_1010.add(inputField_1010);
51         inputPanel_1010.add(setButton_1010);
52
53         // Panel array visual
54         panelArray_1010 = new JPanel();
55         panelArray_1010.setLayout(new FlowLayout());
56
57         // Panel kontrol
58         JPanel controlPanel_1010 = new JPanel();
59         stepButton_1010 = new JButton("Langkah Selanjutnya");
60         resetButton_1010 = new JButton("Reset");
61         stepButton_1010.setEnabled(false);
62         controlPanel_1010.add(stepButton_1010);
63         controlPanel_1010.add(resetButton_1010);
64
65         // Area teks untuk log langkah-langkah
66         stepArea_1010 = new JTextArea(8, 60);
67         stepArea_1010.setEditable(false);
68         stepArea_1010.setFont(new Font("Monospaced", Font.PLAIN,
14));
69         JScrollPane scrollPane_1010 = new
JScrollPane(stepArea_1010);
70
71         // Tambahkan panel ke frame
72         add(inputPanel_1010, BorderLayout.NORTH);
73         add(panelArray_1010, BorderLayout.CENTER);
74         add(controlPanel_1010, BorderLayout.SOUTH);
75         add(scrollPane_1010, BorderLayout.EAST);
76
77         // Event Set Array
78         setButton_1010.addActionListener(e_1010 ->
setArrayFromInput());
79
80         // Event Langkah Selanjutnya
81         stepButton_1010.addActionListener(e_1010 -> performStep());
82
83         // Event Reset
84         resetButton_1010.addActionListener(e_1010 -> reset());
85     }
86
87     private void setArrayFromInput() {
88         String text_1010 = inputField_1010.getText().trim();
89         if (text_1010.isEmpty()) return;
90         String[] parts_1010 = text_1010.split(",");
91         array_1010 = new int[parts_1010.length];

```

```

92         try {
93             for (int k_1010 = 0; k_1010 < parts_1010.length;
k_1010++) {
94                 array_1010[k_1010] =
Integer.parseInt(parts_1010[k_1010].trim());
95             }
96         } catch (NumberFormatException e_1010) {
97             JOptionPane.showMessageDialog(this, "Masukkan hanya
angka "
98                 + "yang dipisahkan koma!", "Error",
JOptionPane.ERROR_MESSAGE);
99             return;
100         }
101         i_1010 = 0;
102         j_1010 = 0;
103         stepCount_1010 = 1;
104         sorting_1010 = true;
105         stepButton_1010.setEnabled(true);
106         stepArea_1010.setText("");
107         panelArray_1010.removeAll();
108         labelArray_1010 = new JLabel[array_1010.length];
109         for (int k_1010 = 0; k_1010 < array_1010.length; k_1010++)
{
110             labelArray_1010[k_1010] = new
JLabel(String.valueOf(array_1010[k_1010]));
111             labelArray_1010[k_1010].setFont(new Font("Arial",
Font.BOLD, 24));
112             labelArray_1010[k_1010].setOpaque(true);
113             labelArray_1010[k_1010].setBackground(Color.WHITE);
114
labelArray_1010[k_1010].setBorder(BorderFactory.createLineBorder(Co
lor.BLACK));
115             labelArray_1010[k_1010].setPreferredSize(new
Dimension(50, 50));
116
labelArray_1010[k_1010].setHorizontalAlignment(SwingConstants.CENTE
R);
117             panelArray_1010.add(labelArray_1010[k_1010]);
118         }
119         panelArray_1010.revalidate();
120         panelArray_1010.repaint();
121     }
122
123     private void performStep() {
124         if (!sorting_1010 || i_1010 >= array_1010.length - 1) {
125             sorting_1010 = false;
126             stepButton_1010.setEnabled(false);
127             JOptionPane.showMessageDialog(this, "Sorting
selesai!");
128             return;
129         }
130         resetHighlights();
131         StringBuilder stepLog_1010 = new StringBuilder();
132         labelArray_1010[j_1010].setBackground(Color.CYAN);

```

```

133     labelArray_1010[j_1010 + 1].setBackground(Color.CYAN);
134     if (array_1010[j_1010] > array_1010[j_1010 + 1]) {
135         // Swap
136         int temp_1010 = array_1010[j_1010];
137         array_1010[j_1010] = array_1010[j_1010 + 1];
138         array_1010[j_1010 + 1] = temp_1010;
139         labelArray_1010[j_1010].setBackground(Color.RED);
140         labelArray_1010[j_1010 + 1].setBackground(Color.RED);
141         stepLog_1010.append("Langkah
142 ").append(stepCount_1010).append(": Menukar elemen ke-")
143         .append(j_1010).append("
144 ") .append(array_1010[j_1010 + 1]).append(" dengan elemen ke-")
145         .append(j_1010 + 1).append("
146 ") .append(array_1010[j_1010]).append(")\n");
147     } else {
148         stepLog_1010.append("Langkah
149 ").append(stepCount_1010).append(": Tidak ada pertukaran antara ke-")
150         .append(j_1010).append("
151 ") .append(array_1010[j_1010]).append(" dan ke-") .append(j_1010 +
152 1).append(" ") .append(array_1010[j_1010 + 1]).append(")\n");
153     }
154     stepLog_1010.append("Hasil:
155 ").append(arrayToString(array_1010)).append("\n\n");
156     stepArea_1010.append(stepLog_1010.toString());
157     updateLabels();
158     j_1010++;
159     if (j_1010 >= array_1010.length - i_1010 - 1) {
160         j_1010 = 0;
161         i_1010++;
162     }
163     stepCount_1010++;
164     if (i_1010 >= array_1010.length - 1) {
165         sorting_1010 = false;
166         stepButton_1010.setEnabled(false);
167         JOptionPane.showMessageDialog(this, "Sorting
168 selesail!");
169     }
170 }
171
172 private void updateLabels() {
173     for (int k_1010 = 0; k_1010 < array_1010.length; k_1010++)
174 {
175     labelArray_1010[k_1010].setText(String.valueOf(array_1010[k_1010]))
176 ;
177     }
178 }
179
180 private void resetHighlights() {
181     for (JLabel label_1010 : labelArray_1010) {
182         label_1010.setBackground(Color.WHITE);
183     }
184 }

```



```

175
176     private void reset() {
177         inputField_1010.setText("");
178         panelArray_1010.removeAll();
179         panelArray_1010.revalidate();
180         panelArray_1010.repaint();
181         stepArea_1010.setText("");
182         stepButton_1010.setEnabled(false);
183         sorting_1010 = false;
184         i_1010 = 0;
185         j_1010 = 0;
186         stepCount_1010 = 1;
187     }
188
189     private String arrayToString(int[] arr_1010) {
190         StringBuilder sb_1010 = new StringBuilder();
191         for (int k_1010 = 0; k_1010 < arr_1010.length; k_1010++) {
192             sb_1010.append(arr_1010[k_1010]);
193             if (k_1010 < arr_1010.length - 1) sb_1010.append(", ");
194         }
195         return sb_1010.toString();
196     }
197
198     public static void main(String[] args_1010) {
199         SwingUtilities.invokeLater(() -> {
200             BubbleSortGUI_2511531010 gui_1010 = new
BubbleSortGUI_2511531010();
201             gui_1010.setVisible(true);
202         });
203     }
204 }

```

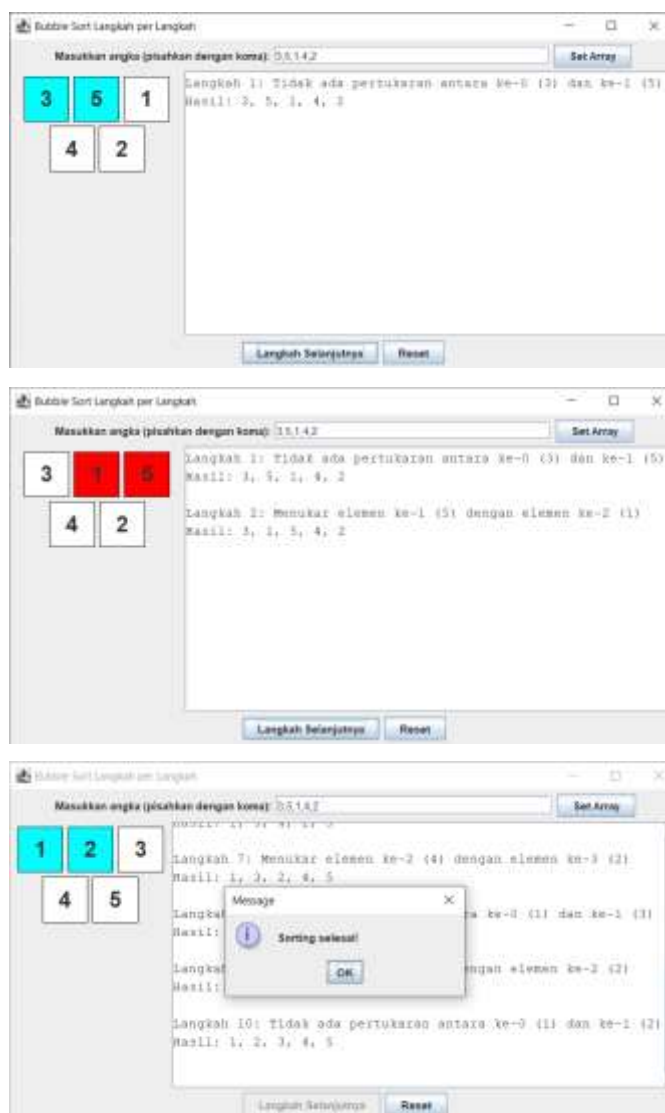
Gambar 2.7

Class ini merupakan implementasi Bubble Sort dalam bentuk GUI menggunakan Java Swing. Program menyediakan input berupa angka yang dipisahkan dengan koma, kemudian menampilkan data dalam bentuk kotak-kotak visual. Saat tombol Langkah Selanjutnya ditekan, program akan menjalankan satu langkah Bubble Sort.

Setiap pasangan elemen yang sedang dibandingkan akan diberi warna biru (cyan). Jika terjadi pertukaran data, kedua elemen akan berubah menjadi warna merah sehingga pengguna dapat mengetahui bagian mana yang sedang diproses. Selain itu, terdapat area teks yang menampilkan keterangan setiap langkah sorting beserta hasil array setelah proses dilakukan.

Program juga menyediakan tombol Reset untuk mengulang proses dari awal dan tombol Set Array untuk memasukkan data baru. Dengan visualisasi ini, pengguna dapat melihat bagaimana Bubble Sort bekerja secara bertahap hingga seluruh data terurut.

2.2.8 Output Bubble Sort GUI



Gambar 2.8

Output program berupa tampilan GUI interaktif yang menunjukkan proses sorting secara langsung. Kotak berwarna biru menandakan elemen yang

sedang dibandingkan, sedangkan kotak berwarna merah menunjukkan bahwa terjadi pertukaran data. Pada area log, pengguna dapat melihat informasi detail mengenai langkah yang sedang dilakukan dan kondisi array setelah proses tersebut. Ketika seluruh data telah tersusun dari nilai terkecil ke terbesar, program akan menampilkan pesan bahwa proses Bubble Sort telah selesai.

2.2.9 Class Merge Sort GUI

```
2  package pekan8_2511531010;
3
4  import java.awt.BorderLayout;
5  import java.awt.Color;
6  import java.awt.Dimension;
7  import java.awt.FlowLayout;
8  import java.awt.Font;
9  import java.util.LinkedList;
10 import java.util.Queue;
11 import javax.swing.BorderFactory;
12 import javax.swing.JButton;
13 import javax.swing.JFrame;
14 import javax.swing.JLabel;
15 import javax.swing.JOptionPane;
16 import javax.swing.JPanel;
17 import javax.swing.JScrollPane;
18 import javax.swing.JTextArea;
19 import javax.swing.JTextField;
20 import javax.swing.SwingConstants;
21 import javax.swing.SwingUtilities;
22
23
24 public class MergeSortGUI_2511531010 extends JFrame {
25     private static final long serialVersionUID = 1L;
26     private int[] array_1010;
27     private JLabel[] labelArray_1010;
28     private JButton stepButton_1010, resetButton_1010,
29     setButton_1010;
30     private JTextField inputField_1010;
31     private JPanel panelArray_1010;
32     private JTextArea stepArea_1010;
33     // Variabel pendukung algoritma langkah demi langkah Merge Sort
34     private Queue<int[]> mergeQueue_1010 = new LinkedList<>();
35     private int stepCount_1010 = 1;
36     private boolean isMerging_1010 = false;
37     private boolean copying_1010 = false;
38
39     private int left_1010, mid_1010, right_1010;
40     private int[] temp_1010;
```

```

41     private int i_1010, j_1010, k_1010;
42
43     /**
44      * Create the frame.
45      */
46
47     public MergeSortGUI_2511531010() {
48         setTitle("Merge Sort Langkah per Langkah");
49         setSize(750, 450);
50         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
51         setLocationRelativeTo(null);
52         setLayout(new BorderLayout());
53
54         // Panel input
55         JPanel inputPanel_1010 = new JPanel(new FlowLayout());
56         inputField_1010 = new JTextField(30);
57         setButton_1010 = new JButton("Set Array");
58         inputPanel_1010.add(new JLabel("Masukkan angka (pisahkan
dengan koma):"));
59         inputPanel_1010.add(inputField_1010);
60         inputPanel_1010.add(setButton_1010);
61
62         // Panel array visual
63         panelArray_1010 = new JPanel();
64         panelArray_1010.setLayout(new FlowLayout());
65
66         // Panel kontrol
67         JPanel controlPanel_1010 = new JPanel();
68         stepButton_1010 = new JButton("Langkah Selanjutnya");
69         resetButton_1010 = new JButton("Reset");
70         stepButton_1010.setEnabled(false);
71         controlPanel_1010.add(stepButton_1010);
72         controlPanel_1010.add(resetButton_1010);
73
74         // Area teks untuk log langkah-langkah
75         stepArea_1010 = new JTextArea(10, 60);
76         stepArea_1010.setEditable(false);
77         stepArea_1010.setFont(new Font("Monospaced", Font.PLAIN,
14));
78         JScrollPane scrollPane_1010 = new
JScrollPane(stepArea_1010);
79
80         // Tambahkan panel ke frame
81         add(inputPanel_1010, BorderLayout.NORTH);
82         add(panelArray_1010, BorderLayout.CENTER);
83         add(controlPanel_1010, BorderLayout.SOUTH);
84         add(scrollPane_1010, BorderLayout.EAST);
85
86         // Event Listeners
87         setButton_1010.addActionListener(e_1010 ->
setArrayFromInput());
88         stepButton_1010.addActionListener(e_1010 -> performStep());
89         resetButton_1010.addActionListener(e_1010 -> reset());
90     }

```

```

91
92     private void generateMergeSteps(int l_1010, int r_1010) {
93         if (l_1010 < r_1010) {
94             int m_1010 = l_1010 + (r_1010 - l_1010) / 2;
95             generateMergeSteps(l_1010, m_1010);
96             generateMergeSteps(m_1010 + 1, r_1010);
97             mergeQueue_1010.add(new int[]{l_1010, m_1010, r_1010});
98         }
99     }
100
101     private void setArrayFromInput() {
102         String text_1010 = inputField_1010.getText().trim();
103         if (text_1010.isEmpty()) return;
104         String[] parts_1010 = text_1010.split(",");
105         array_1010 = new int[parts_1010.length];
106         try {
107             for (int i_idx = 0; i_idx < parts_1010.length; i_idx++)
108             {
109                 array_1010[i_idx] =
110                 Integer.parseInt(parts_1010[i_idx].trim());
111             } catch (NumberFormatException e_1010) {
112                 JOptionPane.showMessageDialog(this, "Masukkan hanya
113                 angka!",
114                 "Error", JOptionPane.ERROR_MESSAGE);
115                 return;
116             }
117
118             labelArray_1010 = new JLabel[array_1010.length];
119             panelArray_1010.removeAll();
120             for (int i_idx = 0; i_idx < array_1010.length; i_idx++) {
121                 labelArray_1010[i_idx] = new
122                 JLabel(String.valueOf(array_1010[i_idx]));
123                 labelArray_1010[i_idx].setFont(new Font("Arial",
124                 Font.BOLD, 24));
125                 labelArray_1010[i_idx].setOpaque(true);
126                 labelArray_1010[i_idx].setBackground(Color.WHITE);
127
128                 labelArray_1010[i_idx].setBorder(BorderFactory.createLineBorder(Color.BLACK));
129                 labelArray_1010[i_idx].setPreferredSize(new
130                 Dimension(50, 50));
131                 labelArray_1010[i_idx].setHorizontalAlignment(SwingConstants.CENTER);
132                 panelArray_1010.add(labelArray_1010[i_idx]);
133             }
134
135             mergeQueue_1010.clear();
136             generateMergeSteps(0, array_1010.length - 1);
137             stepButton_1010.setEnabled(true);
138             stepArea_1010.setText("");
139             stepCount_1010 = 1;
140             isMerging_1010 = false;

```

```

135     panelArray_1010.revalidate();
136     panelArray_1010.repaint();
137 }
138
139 private void performStep() {
140     resetHighlights();
141
142     if (!isMerging_1010 && !mergeQueue_1010.isEmpty()) {
143         int[] range_1010 = mergeQueue_1010.poll();
144         left_1010 = range_1010[0];
145         mid_1010 = range_1010[1];
146         right_1010 = range_1010[2];
147         temp_1010 = new int[right_1010 - left_1010 + 1];
148         i_1010 = left_1010;
149         j_1010 = mid_1010 + 1;
150         k_1010 = 0;
151         copying_1010 = false;
152         isMerging_1010 = true;
153         stepArea_1010.append("Langkah " + stepCount_1010++ +
154             ": Mulai merge dari " + left_1010 + " ke " +
155             right_1010 + "\n");
156         return;
157     }
158
159     if (isMerging_1010 && !copying_1010) {
160         if (i_1010 <= mid_1010 && j_1010 <= right_1010) {
161             labelArray_1010[i_1010].setBackground(Color.CYAN);
162             labelArray_1010[j_1010].setBackground(Color.CYAN);
163             if (array_1010[i_1010] <= array_1010[j_1010]) {
164                 temp_1010[k_1010++] = array_1010[i_1010++];
165             } else {
166                 temp_1010[k_1010++] = array_1010[j_1010++];
167             }
168             stepArea_1010.append("Langkah " + stepCount_1010++
169 + ": Bandingkan dan salin elemen\n");
170             return;
171         } else if (i_1010 <= mid_1010) {
172             temp_1010[k_1010++] = array_1010[i_1010++];
173             stepArea_1010.append("Langkah " + stepCount_1010++
174 + ": Salin sisa kiri\n");
175             return;
176         } else if (j_1010 <= right_1010) {
177             temp_1010[k_1010++] = array_1010[j_1010++];
178             stepArea_1010.append("Langkah " + stepCount_1010++
179 + ": Salin sisa kanan\n");
180             return;
181         } else {
182             copying_1010 = true;
183             k_1010 = 0;
184             return;
185         }
186     }
187
188     if (copying_1010 && k_1010 < temp_1010.length) {

```

```

185         array_1010[left_1010 + k_1010] = temp_1010[k_1010];
186         labelArray_1010[left_1010 +
k_1010].setText(String.valueOf(temp_1010[k_1010]));
187         labelArray_1010[left_1010 +
k_1010].setBackground(Color.GREEN);
188         k_1010++;
189         stepArea_1010.append("Langkah " + stepCount_1010++ + ":
Tempelkan ke array utama\n");
190         return;
191     }
192
193     if (copying_1010 && k_1010 == temp_1010.length) {
194         isMerging_1010 = false;
195         copying_1010 = false;
196     }
197
198     if (mergeQueue_1010.isEmpty() && !isMerging_1010) {
199         stepArea_1010.append("Selesai.\n");
200         stepButton_1010.setEnabled(false);
201         JOptionPane.showMessageDialog(this, "Merge Sort
selesai!");
202     }
203 }
204
205 private void resetHighlights() {
206     if (labelArray_1010 == null) return;
207     for (JLabel label_1010 : labelArray_1010) {
208         label_1010.setBackground(Color.WHITE);
209     }
210 }
211
212 private void reset() {
213     inputField_1010.setText("");
214     panelArray_1010.removeAll();
215     panelArray_1010.revalidate();
216     panelArray_1010.repaint();
217     stepArea_1010.setText("");
218     stepButton_1010.setEnabled(false);
219     mergeQueue_1010.clear();
220     isMerging_1010 = false;
221     stepCount_1010 = 1;
222 }
223
224 public static void main(String[] args_1010) {
225     SwingUtilities.invokeLater(() -> {
226         MergeSortGUI_2511531010 gui_1010 = new
MergeSortGUI_2511531010();
227         gui_1010.setVisible(true);
228     });
229 }
230 }

```

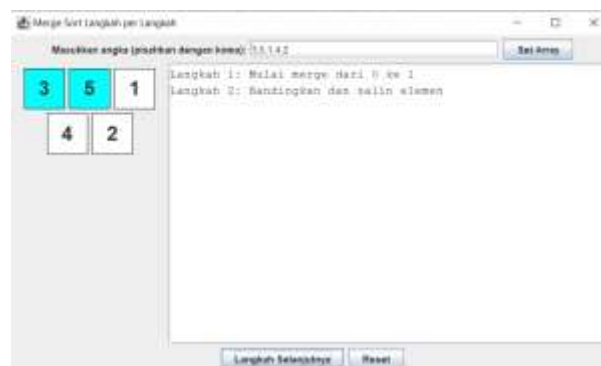
Gambar 2.9

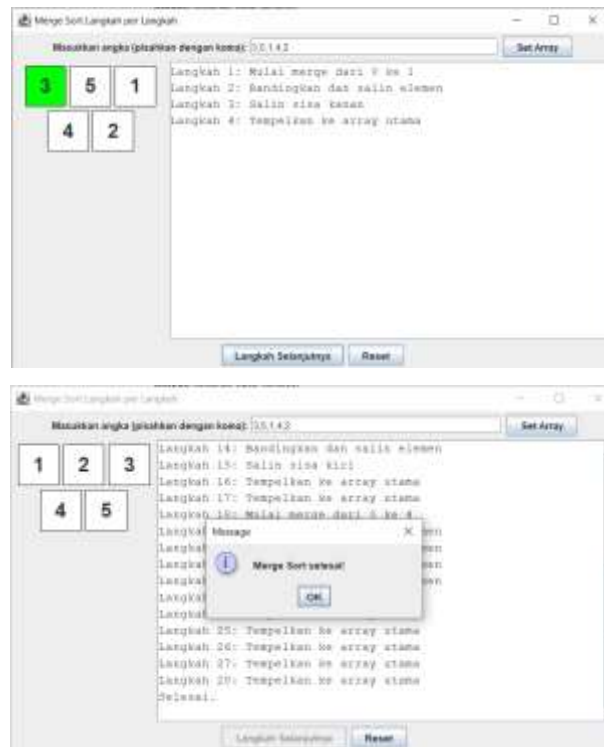
Class ini merupakan implementasi Bubble Sort dalam bentuk GUI menggunakan Java Swing. Program menyediakan input berupa angka yang dipisahkan dengan koma, kemudian menampilkan data dalam bentuk kotak-kotak visual. Saat tombol Langkah Selanjutnya ditekan, program akan menjalankan satu langkah Bubble Sort.

Setiap pasangan elemen yang sedang dibandingkan akan diberi warna biru (cyan). Jika terjadi pertukaran data, kedua elemen akan berubah menjadi warna merah sehingga pengguna dapat mengetahui bagian mana yang sedang diproses. Selain itu, terdapat area teks yang menampilkan keterangan setiap langkah sorting beserta hasil array setelah proses dilakukan.

Program juga menyediakan tombol Reset untuk mengulang proses dari awal dan tombol Set Array untuk memasukkan data baru. Dengan visualisasi ini, pengguna dapat melihat bagaimana Bubble Sort bekerja secara bertahap hingga seluruh data terurut.

2.2.10 Output Merge Sort GUI





Gambar 2.10

Output berupa tampilan GUI yang memperlihatkan proses penggabungan data secara bertahap. Elemen yang sedang dibandingkan akan diberi warna biru, sedangkan elemen yang berhasil ditempatkan pada posisi yang benar akan berubah menjadi warna hijau. Pada area log, pengguna dapat melihat keterangan mengenai proses merge yang sedang berlangsung, mulai dari pembagian data hingga penggabungan kembali. Setelah semua langkah selesai, seluruh data tampil dalam keadaan terurut dan program memberikan notifikasi bahwa proses Merge Sort telah berhasil diselesaikan.

BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa algoritma sorting memiliki peran yang sangat penting dalam pengolahan data karena mampu menyusun data menjadi lebih teratur sehingga lebih mudah dikelola dan diproses. Melalui implementasi algoritma Shell Sort, Quick Sort, dan Merge Sort, dapat dipahami bahwa setiap metode memiliki cara kerja dan strategi yang berbeda dalam menghasilkan data yang terurut, namun tetap memiliki tujuan yang sama yaitu meningkatkan efisiensi pengolahan data.

Praktikum ini juga memberikan pemahaman bahwa pemilihan algoritma sorting yang tepat dapat memengaruhi kinerja program, terutama ketika jumlah data yang diolah semakin besar. Shell Sort mampu mempercepat proses pengurutan dengan penggunaan gap, Quick Sort memanfaatkan teknik divide and conquer untuk membagi permasalahan menjadi bagian yang lebih kecil, sedangkan Merge Sort bekerja dengan membagi dan menggabungkan data secara sistematis hingga menghasilkan urutan yang benar.

Selain itu, penggunaan GUI pada Bubble Sort dan Merge Sort memberikan pengalaman belajar yang lebih interaktif karena proses sorting dapat diamati secara visual dan bertahap. Melalui visualisasi tersebut, konsep perbandingan, pertukaran, pembagian, dan penggabungan data menjadi lebih mudah dipahami dibandingkan hanya melihat hasil akhir program.

DAFTAR PUSTAKA

- [1] Oracle Corporation. 2023. *Java Documentation*. Diakses dari <https://docs.oracle.com/javase/8/docs/>
- [2] Oracle Corporation. 2023. *Creating a GUI With Swing*. Diakses dari <https://docs.oracle.com/javase/tutorial/uiswing/>

TUGAS PRAKTIKUM

Soal:

Topik: Algoritma Sorting (Shell, Quick, dan Merge)

Tema: Mengurutkan Playlist Lagu

1. Deskripsi Tugas

Mahasiswa diminta mengimplementasikan salah SATU algoritma sorting untuk mengurutkan data lagu dalam playlist. Pilih salah satu: Shell Sort, Quick Sort, atau Merge Sort. Data lagu disimpan dalam array (bukan linked list). Program harus bisa mengurutkan berdasarkan Judul (A-Z) atau Durasi (terpendek ke terpanjang).

2. Aturan Penamaan (Wajib)

1. Nama Kelas: gunakan NIM lengkap. Contoh: Sorting_2411532014
2. Nama variabel & method: wajib pakai 4 digit terakhir NIM. Contoh: dataLagu_2014, shellSort_2014()
3. Jika memilih Quick Sort, method utama harus quickSort_xxxx(), jika Merge: mergeSort_xxxx(), jika Shell: shellSort_xxxx()

3. Struktur Program

Kelas Lagu (sederhana)

Atribut: judul (String), penyanyi (String), durasi (int)

Kelas Sorting_NIM

1. Minimal harus ada:
2. Array dataLagu_xxxx untuk menyimpan maksimal 20 lagu
3. Method inputData_xxxx() – untuk mengisi data awal (minimal 7 lagu)
4. Method pilihAlgoritma – cukup implementasikan SATU dari tiga di bawah:\
 - a. shellSort_xxxx() – urutkan berdasarkan judul A-Z
 - b. quickSort_xxxx() – urutkan berdasarkan durasi ascending
 - c. mergeSort_xxxx() – urutkan berdasarkan judul A-Z
5. Method tampilData_xxxx() – menampilkan sebelum dan sesudah sorting

4. Ketentuan Pilihan

Pilih HANYA 1 algoritma. Tulis di laporan alasan memilih algoritma tersebut (4-5 kalimat).

Tidak perlu membuat ketiganya, fokus pada implementasi yang benar dan sesuai aturan penamaan.

5. Contoh Output

== Sorting Playlist NIM: 2411532014 ==

Pilih Algoritma (1=Shell, 2=Quick, 3=Merge): 2

Data Sebelum Sorting:

1. Mio Cristo Piange Diamanti - 270 detik
2. La Rumba Del Perdon - 252 detik
3. La Perla - 196 detik

Data Setelah Quick Sort (Durasi Asc):

1. La Perla - 196 detik
2. La Rumba Del Perdon - 252 detik
3. Mio Cristo Piange Diamanti - 270 detik

6. Kriteria Penilaian

1. Algoritma berjalan benar sesuai pilihan
2. Aturan penamaan NIM 100% dipatuhi
3. Sorting bisa untuk minimal 7 data
4. Menampilkan data sebelum dan sesudah
5. Laporan menjelaskan pilihan algoritma dan kompleksitasnya

Kode Pemograman:

1. Class Lagu_2511531010

```
2. package pekan8_2511531010;
3.
4. public class Lagu_2511531010 {
5.     String judul_1010;
6.     String penyanyi_1010;
7.     int durasi_1010;
8.
9.     public Lagu_2511531010(String judul_1010, String
    penyanyi_1010, int durasi_1010) {
10.         this.judul_1010 = judul_1010;
11.         this.penyanyi_1010 = penyanyi_1010;
12.         this.durasi_1010 = durasi_1010;
13.     }
14. }
```

2. Class Sorting_2511531010

```
1. package pekan8_2511531010;
2.
3. import java.util.Scanner;
4.
5. public class Sorting_2511531010 {
6.
7.     Lagu_2511531010[] dataLagu_1010 = new Lagu_2511531010[20];
8.     int jumlah_1010 = 0;
9.
10.    void inputData_1010() {
11.        dataLagu_1010[jumlah_1010++] = new Lagu_2511531010("Me
    and my", "Blackpink", 210);
12.        dataLagu_1010[jumlah_1010++] = new
    Lagu_2511531010("Lifestyle", "Lisa", 200);
13.        dataLagu_1010[jumlah_1010++] = new Lagu_2511531010("Damn
    Right", "Jennie", 190);
14.        dataLagu_1010[jumlah_1010++] = new
    Lagu_2511531010("Earthquake", "Jisoo", 180);
15.        dataLagu_1010[jumlah_1010++] = new Lagu_2511531010("Dance
    all night", "Rose", 220);
16.        dataLagu_1010[jumlah_1010++] = new
    Lagu_2511531010("Gameboy", "Rose", 230);
17.        dataLagu_1010[jumlah_1010++] = new
    Lagu_2511531010("ExtraL", "Jennie", 240);
18.    }
19.
20.    void tampilData_1010() {
21.        for (int i_1010 = 0; i_1010 < jumlah_1010; i_1010++) {
22.            System.out.println((i_1010 + 1) + ". "
23.                + dataLagu_1010[i_1010].judul_1010 + " - "
```

```

24.         + dataLagu_1010[i_1010].durasi_1010 + "
    detik");
25.     }
26. }
27.
28. //shell sort
29. void shellSort_1010() {
30.     for (int gap_1010 = jumlah_1010 / 2; gap_1010 > 0;
    gap_1010 /= 2) {
31.         for (int i_1010 = gap_1010; i_1010 < jumlah_1010;
    i_1010++) {
32.
33.             Lagu_2511531010 temp_1010 =
    dataLagu_1010[i_1010];
34.             int j_1010 = i_1010;
35.
36.             while (j_1010 >= gap_1010 &&
    dataLagu_1010[j_1010 -
37. gap_1010].judul_1010
    .compareToIgnoreCase(temp_1010.judul_1010)
38. > 0) {
39.
40.                 dataLagu_1010[j_1010] = dataLagu_1010[j_1010
    - gap_1010];
41.                 j_1010 -= gap_1010;
42.             }
43.
44.             dataLagu_1010[j_1010] = temp_1010;
45.         }
46.     }
47. }
48.
49. //quick sort
50. void quickSort_1010(int low_1010, int high_1010) {
51.     if (low_1010 < high_1010) {
52.         int pi_1010 = partition_1010(low_1010, high_1010);
53.
54.         quickSort_1010(low_1010, pi_1010 - 1);
55.         quickSort_1010(pi_1010 + 1, high_1010);
56.     }
57. }
58.
59. int partition_1010(int low_1010, int high_1010) {
60.
61.     int pivot_1010 = dataLagu_1010[high_1010].durasi_1010;
62.     int i_1010 = low_1010 - 1;
63.
64.     for (int j_1010 = low_1010; j_1010 < high_1010; j_1010++)
    {
65.
66.         if (dataLagu_1010[j_1010].durasi_1010 < pivot_1010) {
67.
68.             i_1010++;
69.

```

```

70.         Lagu_2511531010 temp_1010 =
dataLagu_1010[i_1010];
71.         dataLagu_1010[i_1010] = dataLagu_1010[j_1010];
72.         dataLagu_1010[j_1010] = temp_1010;
73.     }
74. }
75.
76.     Lagu_2511531010 temp_1010 = dataLagu_1010[i_1010 + 1];
77.     dataLagu_1010[i_1010 + 1] = dataLagu_1010[high_1010];
78.     dataLagu_1010[high_1010] = temp_1010;
79.
80.     return i_1010 + 1;
81. }
82.
83. //merge sort
84. void mergeSort_1010(int left_1010, int right_1010) {
85.     if (left_1010 < right_1010) {
86.
87.         int mid_1010 = (left_1010 + right_1010) / 2;
88.
89.         mergeSort_1010(left_1010, mid_1010);
90.         mergeSort_1010(mid_1010 + 1, right_1010);
91.
92.         merge_1010(left_1010, mid_1010, right_1010);
93.     }
94. }
95.
96. void merge_1010(int left_1010, int mid_1010, int right_1010)
{
97.
98.     int n1_1010 = mid_1010 - left_1010 + 1;
99.     int n2_1010 = right_1010 - mid_1010;
100.
101.     Lagu_2511531010[] L_1010 = new
Lagu_2511531010[n1_1010];
102.     Lagu_2511531010[] R_1010 = new
Lagu_2511531010[n2_1010];
103.
104.     for (int i_1010 = 0; i_1010 < n1_1010; i_1010++)
105.         L_1010[i_1010] = dataLagu_1010[left_1010 +
i_1010];
106.
107.     for (int j_1010 = 0; j_1010 < n2_1010; j_1010++)
108.         R_1010[j_1010] = dataLagu_1010[mid_1010 + 1 +
j_1010];
109.
110.     int i_1010 = 0, j_1010 = 0, k_1010 = left_1010;
111.
112.     while (i_1010 < n1_1010 && j_1010 < n2_1010) {
113.
114.         if
(L_1010[i_1010].judul_1010.compareToIgnoreCase(R_1010[j_1010].jud
ul_1010) <= 0)
115.             dataLagu_1010[k_1010++] = L_1010[i_1010++];

```

```

116.         else
117.             dataLagu_1010[k_1010++] = R_1010[j_1010++];
118.     }
119.
120.     while (i_1010 < n1_1010)
121.         dataLagu_1010[k_1010++] = L_1010[i_1010++];
122.
123.     while (j_1010 < n2_1010)
124.         dataLagu_1010[k_1010++] = R_1010[j_1010++];
125. }
126.
127. public static void main(String[] args_1010) {
128.
129.     Scanner input_1010 = new Scanner(System.in);
130.     Sorting_2511531010 sorting_1010 = new
Sorting_2511531010();
131.
132.     sorting_1010.inputData_1010();
133.
134.     System.out.println("=== Sorting Playlist NIM:
2511531010 ===");
135.     System.out.println("1. Shell Sort");
136.     System.out.println("2. Quick Sort");
137.     System.out.println("3. Merge Sort");
138.     System.out.print("Pilih Algoritma: ");
139.
140.     int pilihan_1010 = input_1010.nextInt();
141.
142.     System.out.println("\nData Sebelum Sorting:");
143.     sorting_1010.tampilData_1010();
144.
145.     switch (pilihan_1010) {
146.
147.         case 1:
148.             sorting_1010.shellSort_1010();
149.             System.out.println("\nData Setelah Shell
Sort (Judul A-Z):");
150.             break;
151.
152.         case 2:
153.             sorting_1010.quickSort_1010(0,
sorting_1010.jumlah_1010 - 1);
154.             System.out.println("\nData Setelah Quick
Sort (Durasi Asc):");
155.             break;
156.
157.         case 3:
158.             sorting_1010.mergeSort_1010(0,
sorting_1010.jumlah_1010 - 1);
159.             System.out.println("\nData Setelah Merge
Sort (Judul A-Z):");
160.             break;
161.
162.         default:

```

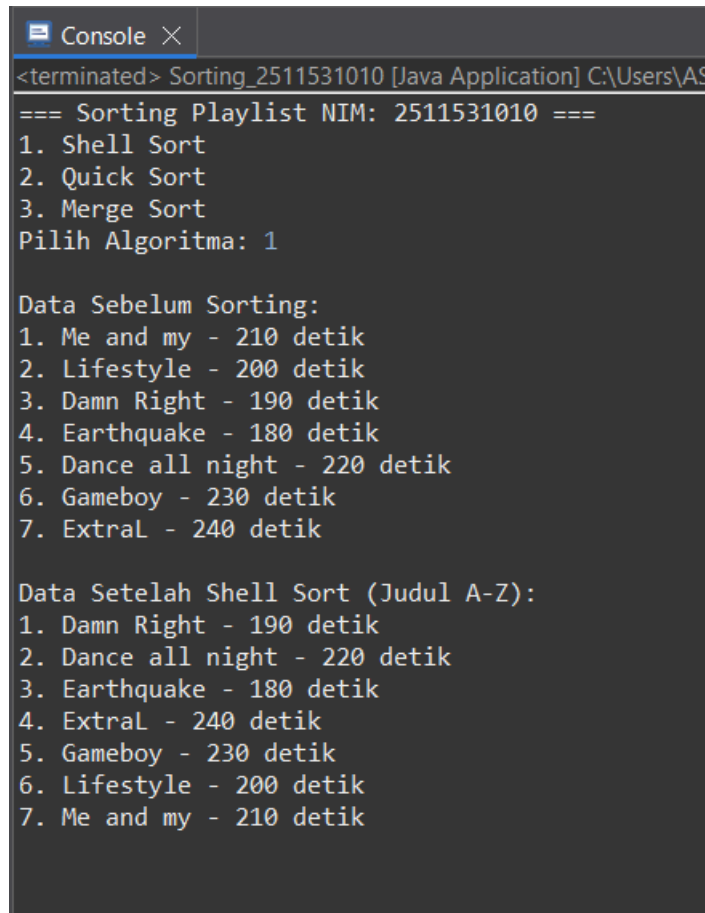


```

163.                                     System.out.println("Pilihan tidak valid!");
164.                                     return;
165.                                 }
166.                                 sorting_1010.tampilData_1010();
167.                             }
168.                         }

```

3. Output



```

Console X
<terminated> Sorting_2511531010 [Java Application] C:\Users\AS
=== Sorting Playlist NIM: 2511531010 ===
1. Shell Sort
2. Quick Sort
3. Merge Sort
Pilih Algoritma: 1

Data Sebelum Sorting:
1. Me and my - 210 detik
2. Lifestyle - 200 detik
3. Damn Right - 190 detik
4. Earthquake - 180 detik
5. Dance all night - 220 detik
6. Gameboy - 230 detik
7. ExtraL - 240 detik

Data Setelah Shell Sort (Judul A-Z):
1. Damn Right - 190 detik
2. Dance all night - 220 detik
3. Earthquake - 180 detik
4. ExtraL - 240 detik
5. Gameboy - 230 detik
6. Lifestyle - 200 detik
7. Me and my - 210 detik

```

Alasan Memilih Algoritma Shell Sort (1)

Pada praktikum ini, saya memilih menggunakan algoritma Shell Sort karena algoritma ini merupakan pengembangan dari Insertion Sort yang memiliki performa lebih baik dalam mengurutkan data. Shell Sort bekerja dengan membandingkan elemen yang memiliki jarak tertentu (gap), sehingga proses perpindahan data dapat dilakukan lebih cepat dibandingkan Insertion Sort biasa. Selain itu, implementasi Shell Sort relatif mudah dipahami dan tidak memerlukan penggunaan memori tambahan yang besar. Algoritma ini

juga cukup efisien untuk mengurutkan data dalam jumlah sedang, sehingga cocok digunakan untuk mengurutkan data playlist lagu pada tugas praktikum ini. Dengan menggunakan Shell Sort, proses pengurutan judul lagu dari A sampai Z dapat dilakukan dengan lebih cepat dan rapi.